

Construye tus primeros modelos de Machine Learning

Regresión Lineal, Regresión Logística, Redes Neuronales y Scikit-Learn

DE LOS DATOS A LAS PREDICCIONES



Agenda del módulo

01

¿Qué es Machine Learning?

Cómo aprende un modelo a partir de ejemplos

02

Regresión Lineal

Predecir valores continuos como precios

03

Regresión Logística

Clasificar con probabilidades

04

Redes Neuronales

Neuronas, capas y entrenamiento

05

Scikit-Learn y flujo práctico

Documentación, Iris y respuestas clave

La idea central del módulo

Machine Learning = descubrir patrones ocultos en los datos para predecir sobre información nueva. No se programan todas las reglas: el algoritmo las aprende solo.



La analogía del mapa del tesoro

Datos

Las pistas del mapa que guían el camino

Algoritmo

El explorador que descubre el patrón

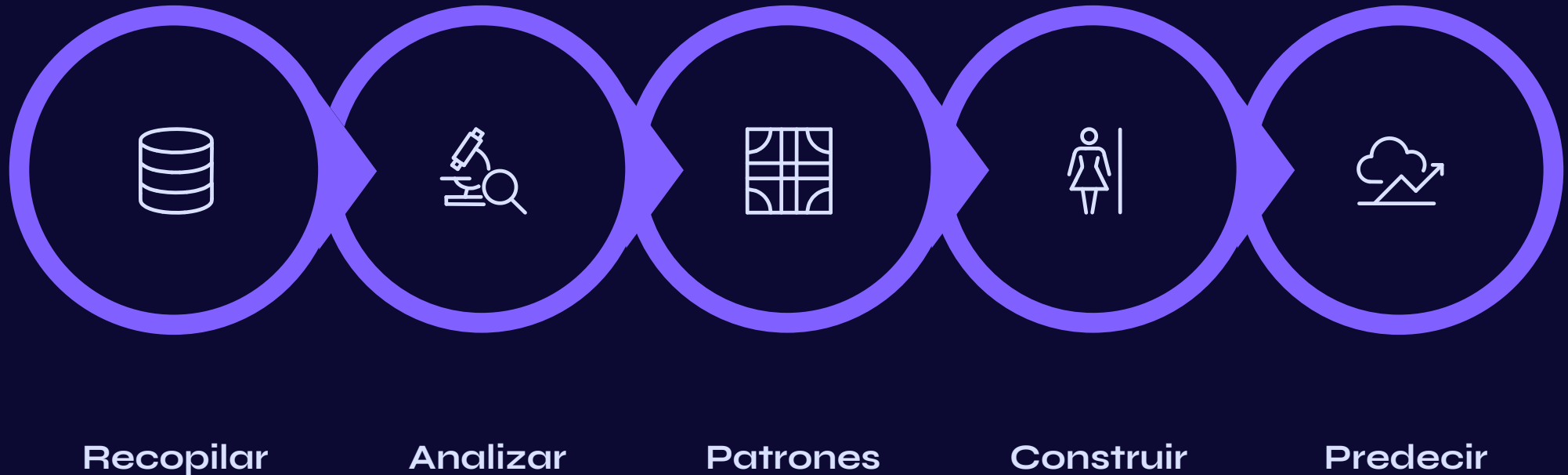
Modelo

El resultado del aprendizaje obtenido

Predicciones

El tesoro: las respuestas sobre datos nuevos

¿Cómo aprende un modelo?



El modelo **no memoriza respuestas**: aprende las relaciones entre variables para generalizar a datos que nunca ha visto antes.



Ejemplo práctico: preferencias de café

¿Comprará el cliente una nueva mezcla? El modelo analiza sus **compras anteriores**, preferencias e historial de pedidos para calcular una **probabilidad de compra**.

- ❏ Cuantos más datos tenga el modelo, más clara será la imagen — como un rompecabezas con más piezas.

Herramientas: Python + Scikit-Learn

Python permite

- Cargar y limpiar datos
- Entrenar modelos
- Realizar predicciones
- Evaluar resultados

Scikit-Learn: la navaja suiza del ML

- Regresión Lineal y Logística
- Árboles de decisión y Redes Neuronales
- División de datos y métricas
- Preprocesamiento integrado

¿Para qué sirve la Regresión Lineal?



Tarea: predecir valores continuos

Caso típico: **precio de viviendas**

- Dormitorios y superficie
- Ubicación del inmueble
- Otras características del hogar

La salida **y** es siempre un número real, no una categoría.

Paso 0: preparación de datos con Pandas

Cargar y explorar

Leer el CSV, revisar estadísticas básicas y conocer la estructura del dataset.

Limpiar y transformar

Gestionar **valores faltantes** y crear nuevas características con conocimiento del dominio.

Regla de oro

Primero preparar los datos, *después* modelar. Un dato sucio rompe el modelo.

División Train / Test

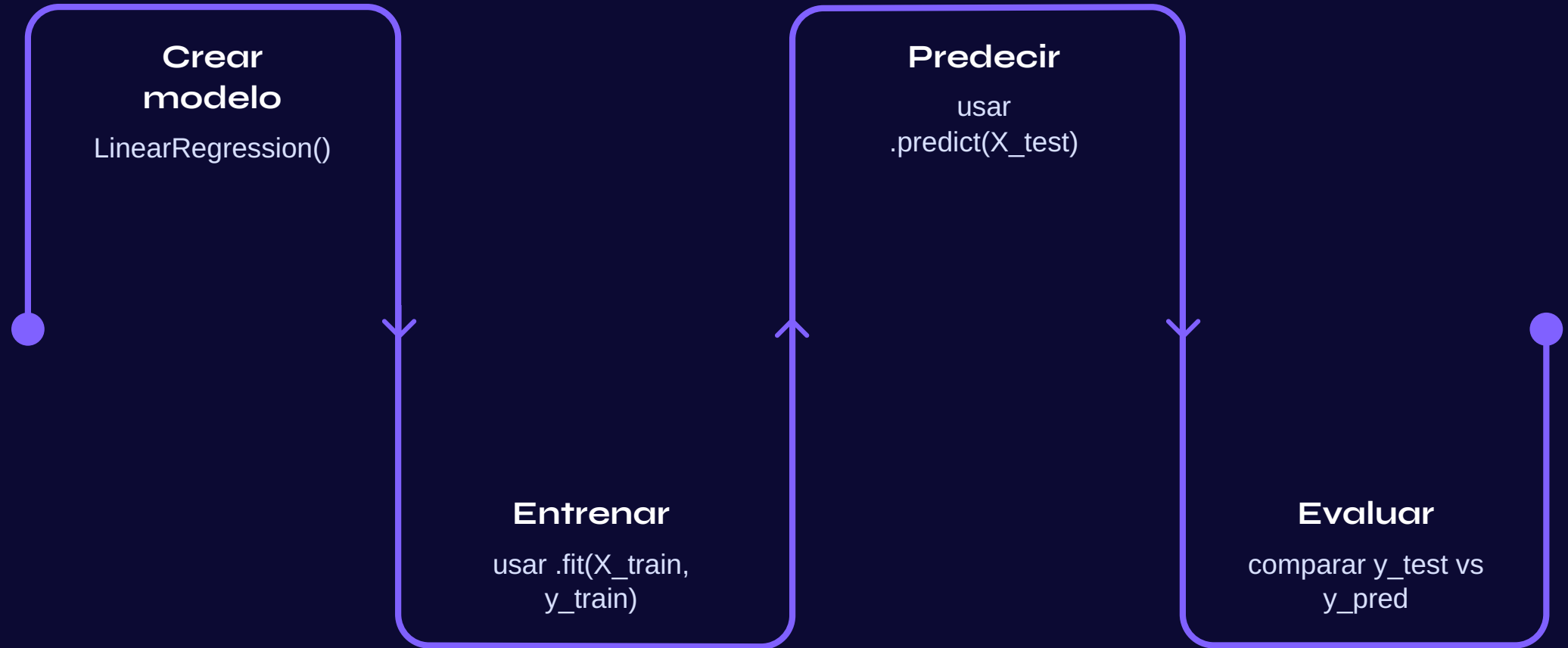


¿Por qué dividir?

El **training set** entrena el modelo. El **test set** evalúa cómo se comporta con datos que *nunca ha visto*.

Herramienta: `train_test_split()` con `random_state=42` para resultados reproducibles.

Flujo completo de Regresión Lineal



La visualización final con `scatter(y_test, y_pred)` permite ver visualmente cuán cerca están las predicciones de los valores reales.

Métricas de Regresión Lineal

MSE — Error Cuadrático Medio

Mide el error promedio al cuadrado entre predicción y valor real.

Cuanto más bajo, mejor. ↓

R^2 — R-cuadrado

Indica qué proporción de la varianza captura el modelo.

Cuanto más cercano a 1, mejor. ↑



No los confundas: MSE ↓ es mejor, R^2 ↑ es mejor. Son complementarios, no opuestos.

Bibliotecas para Regresión Lineal



pandas / numpy

Carga, limpieza, transformación y operaciones numéricas sobre los datos.



sklearn

`LinearRegression`, `train_test_split`,
`mean_squared_error`, `r2_score`



matplotlib

Visualización del scatter plot:
predicciones vs valores reales.

Código: Regresión Lineal completo

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df = pd.read_csv("housing.csv")
X = df[["bedrooms", "sqft", "location"]]
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

modelo = LinearRegression()
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

CLASIFICACIÓN

¿Qué es la clasificación?

 Spam / No spam

Filtrar correos no deseados automáticamente

 Fraude / No fraude

Detectar transacciones sospechosas en tiempo real

 Enfermo / Sano

Diagnóstico asistido por ML en medicina

 Aprueba / No aprueba

Predicción de resultados académicos o crediticios

Cuando solo hay **dos clases posibles**, se llama **clasificación binaria**.

Clasifica con probabilidades

La función sigmoidea

Transforma cualquier valor numérico en una **probabilidad entre 0 y 1** con su característica forma de "S".

Con un **threshold de 0.5**:

- $P > 0.5 \rightarrow$ **Clase 1**
- $P \leq 0.5 \rightarrow$ **Clase 0**

¿Por qué "Regresión" si clasifica?

A pesar del nombre, la Regresión Logística se usa casi siempre para **clasificación**, especialmente binaria. La salida es una probabilidad, no una clase directa.

Ejemplo: detección de spam

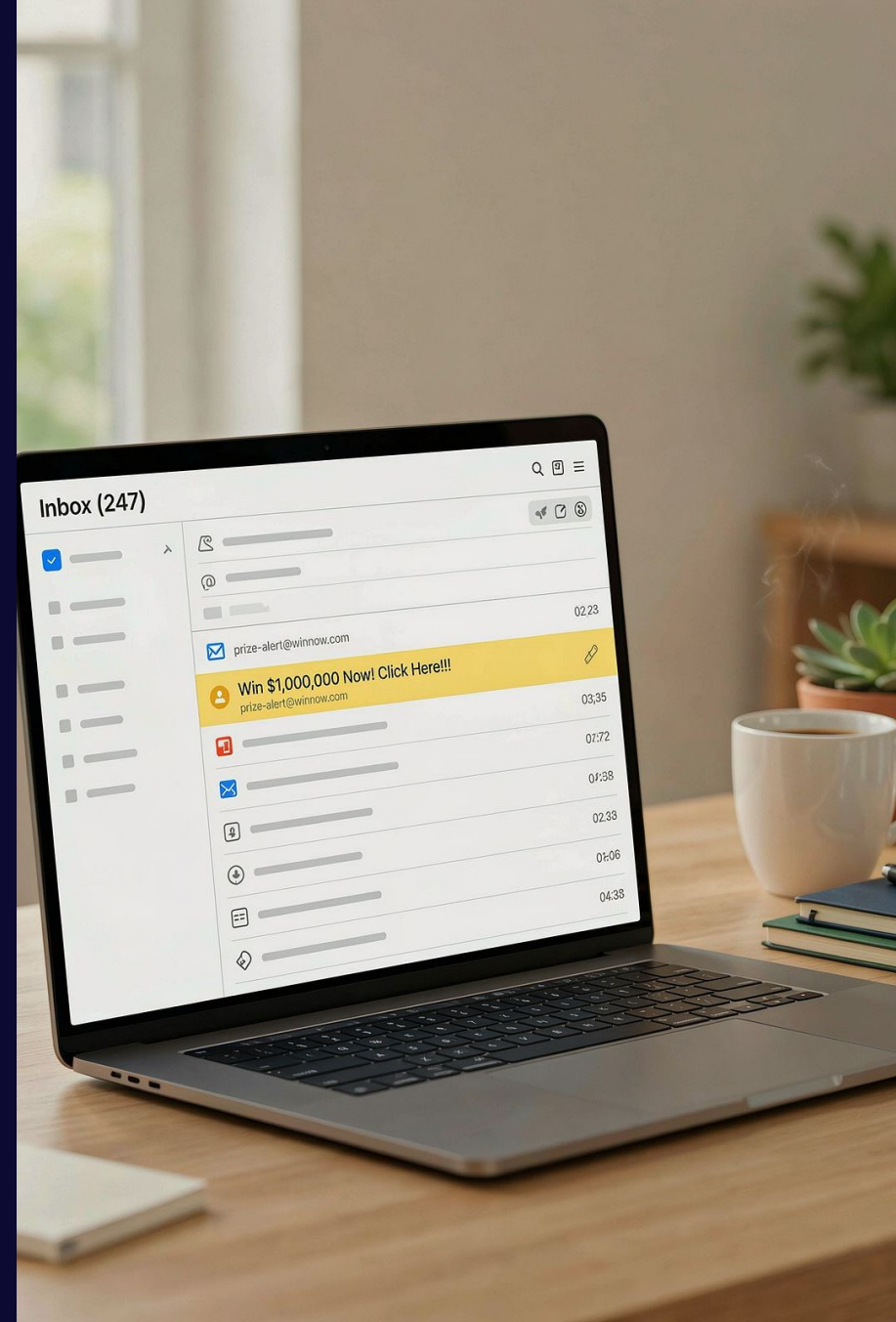
El modelo evalúa características del correo para calcular la probabilidad de ser spam:

Entradas del modelo

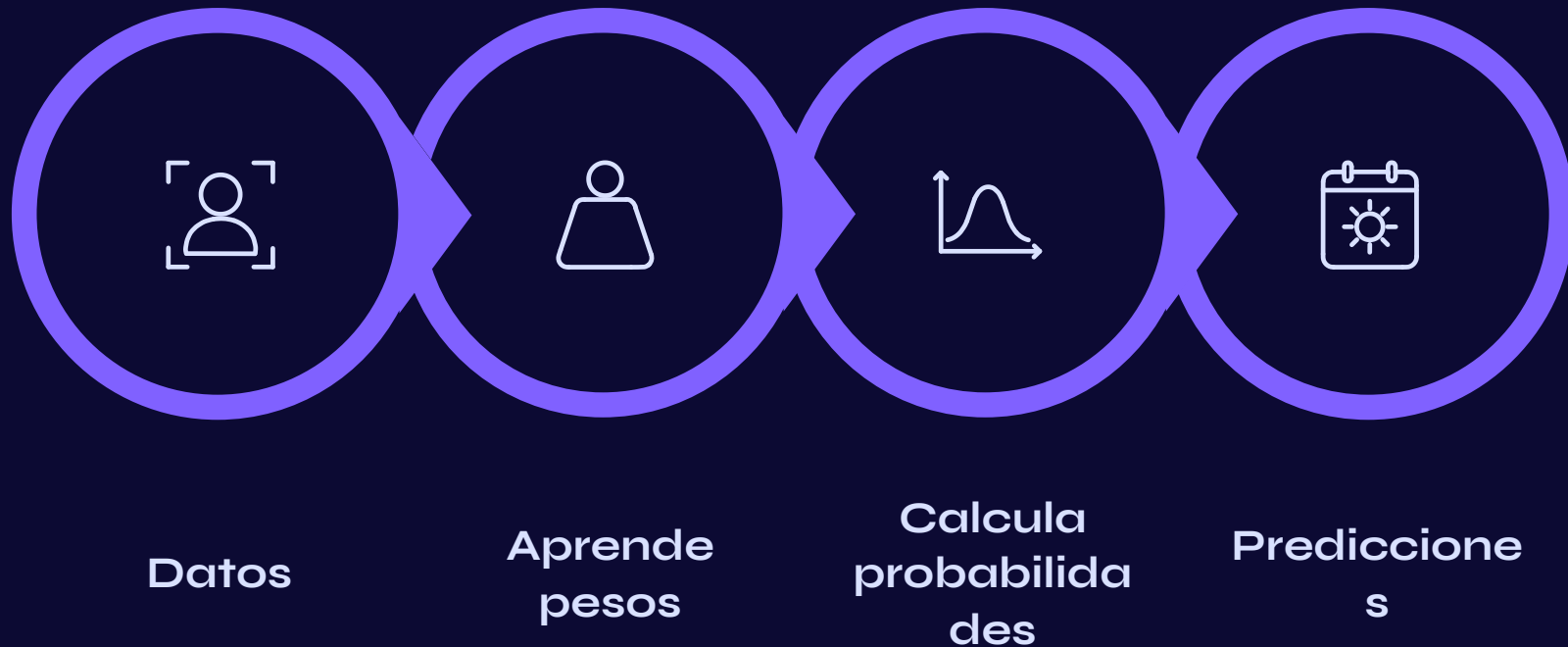
Cantidad de enlaces, palabras sospechosas, remitente, frecuencia de términos

Salida con umbral 0.5

$P(\text{spam}) = 0.87 \rightarrow \text{SPAM}$ | $P(\text{spam}) = 0.15 \rightarrow \text{NO SPAM}$

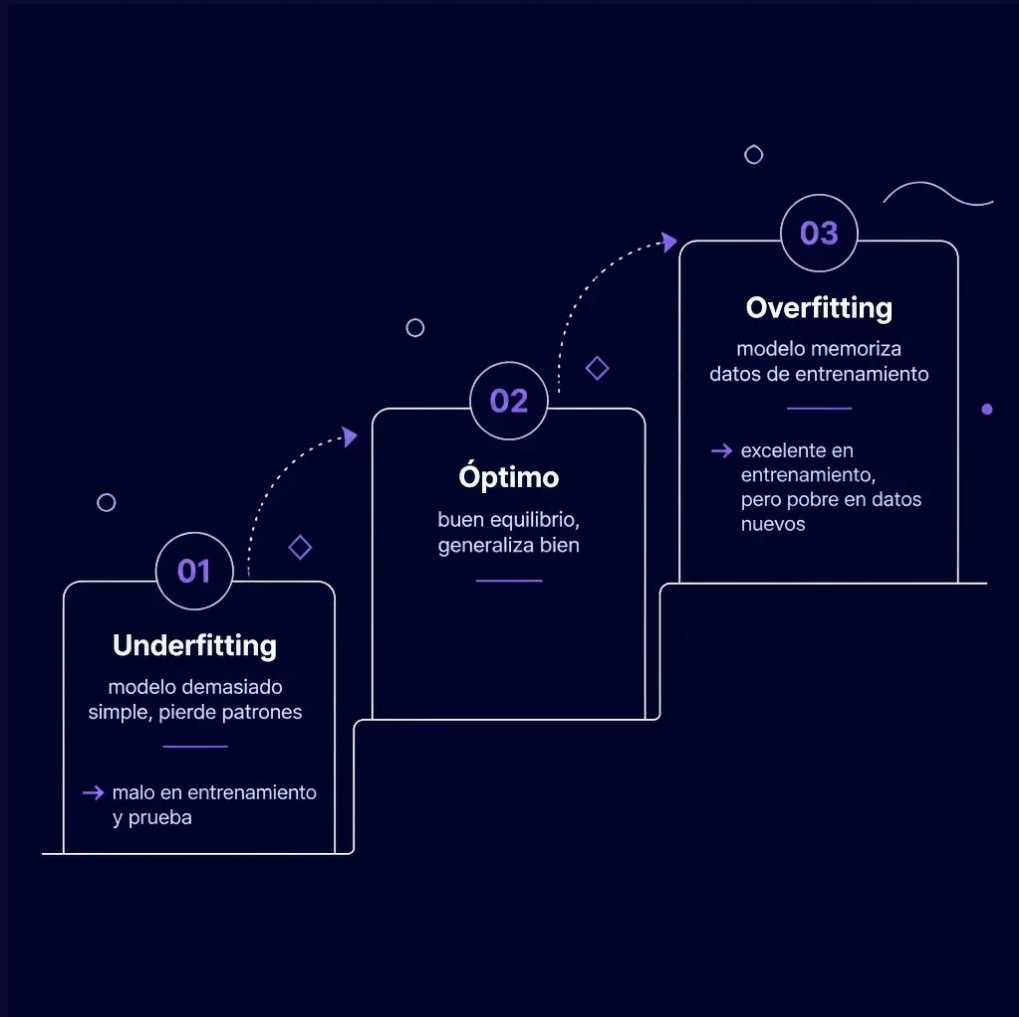


Entrenamiento: cómo aprende la Regresión Logística



Los **pesos** determinan la importancia de cada característica. Un peso mayor implica mayor influencia. El algoritmo los aprende solo — no se definen a mano.

Overfitting vs Underfitting



El equilibrio es el objetivo

- **Underfitting:** modelo demasiado simple, no captura patrones
- **Overfitting:** memoriza en vez de aprender, falla con datos nuevos
- **Punto medio:** generaliza bien a datos no vistos

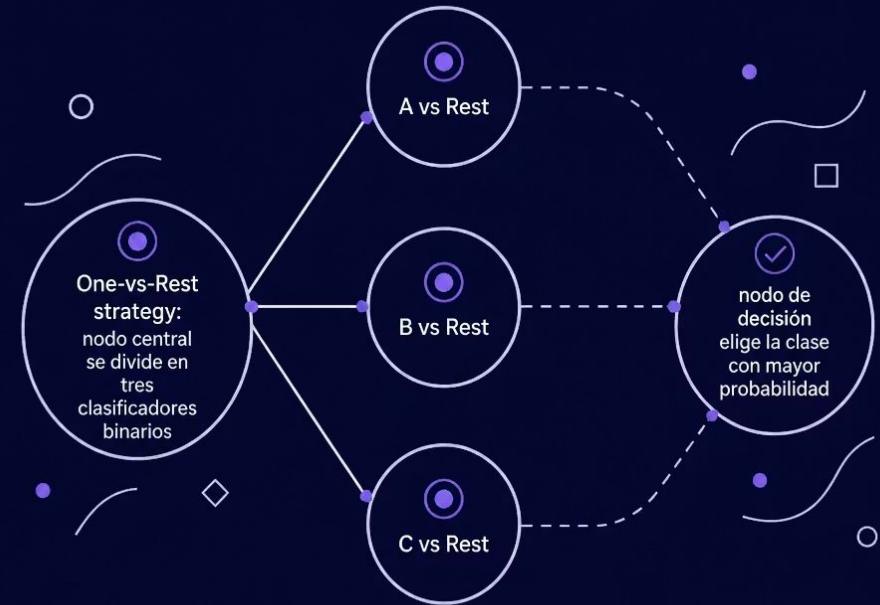
Regresión Logística para múltiples clases

Estrategia: One-vs-Rest

Para 3 clases A, B, C se entrenan **3 modelos binarios** independientes:

- Modelo 1: A vs (B + C)
- Modelo 2: B vs (A + C)
- Modelo 3: C vs (A + B)

La **clase ganadora** es la que obtiene la mayor probabilidad.



Código: Regresión Logística en Scikit-Learn

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

modelo = LogisticRegression()
modelo.fit(X_train, y_train)

y_pred = modelo.predict(X_test) # clase predicha
probabilidades = modelo.predict_proba(X_test) # probabilidad por clase
```

Regresión Lineal vs Regresión Logística

	Regresión Lineal	Regresión Logística
Tarea	Regresión	Clasificación
Predice	Valores continuos	Probabilidades → clases
Ejemplo	Precio de vivienda	Spam / No spam
Salida	Número real	Clase (0 ó 1)
Métricas	MSE, R ²	Accuracy, Precision, Recall, F1

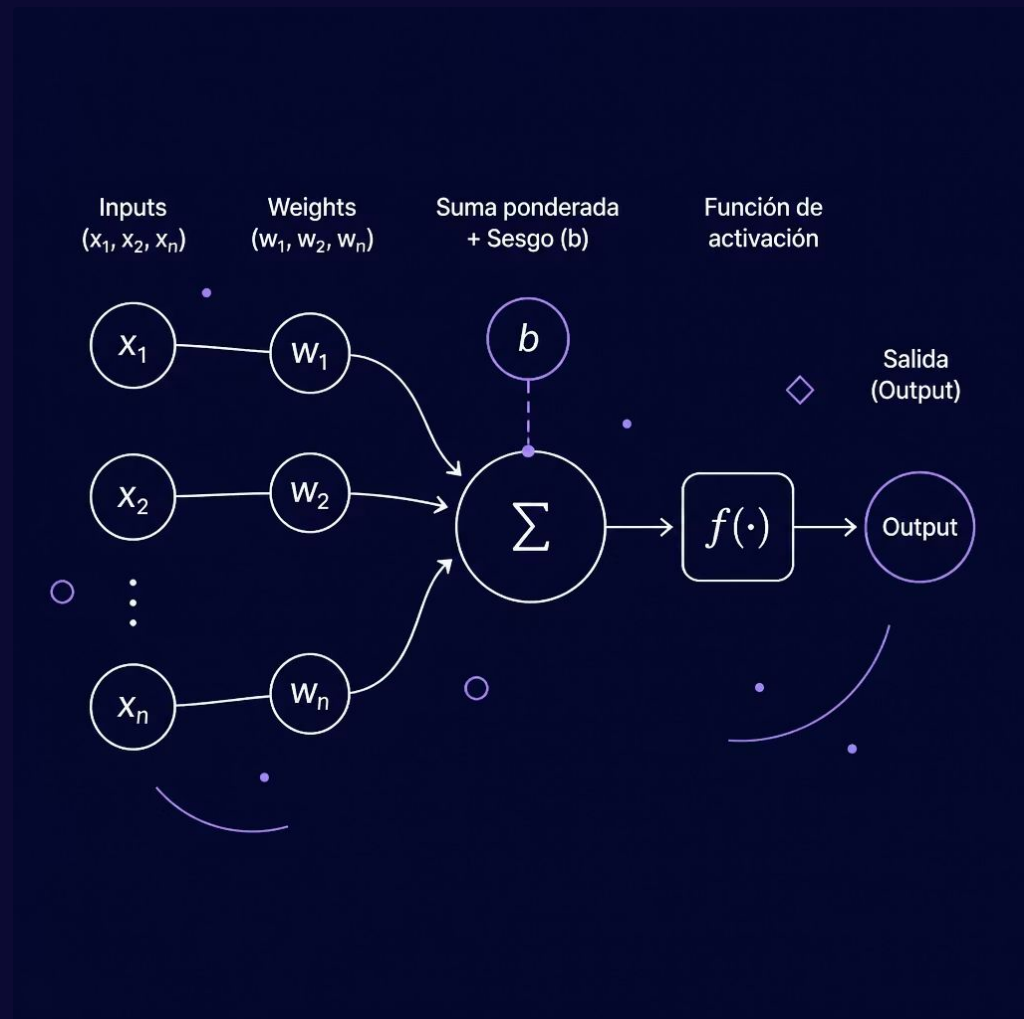


REDES NEURONALES

¿Qué son las Redes Neuronales?

Modelos computacionales **inspirados en el cerebro humano** que aprenden patrones directamente de los datos, sin reglas programadas manualmente.

La neurona como unidad básica



Componentes clave

- **Entradas (x):** información que recibe
- **Pesos (w):** importancia de cada entrada
- **Bias (b):** ajuste independiente
- **Función de activación:** introduce no linealidad

Fórmula conceptual de una neurona

$$z = x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_n \cdot w_n + b$$

$$\text{salida} = f_{\text{activación}}(z)$$

- ❏ Los pesos **w** y el bias **b** no se definen a mano — el modelo los **aprende durante el entrenamiento** ajustándolos iterativamente.

¿Por qué necesitamos función de activación?

Sin activación

La red sería solo una combinación lineal simple — incapaz de aprender relaciones complejas.

Con activación

Se introduce **no linealidad**, permitiendo modelar patrones mucho más complejos y realistas.

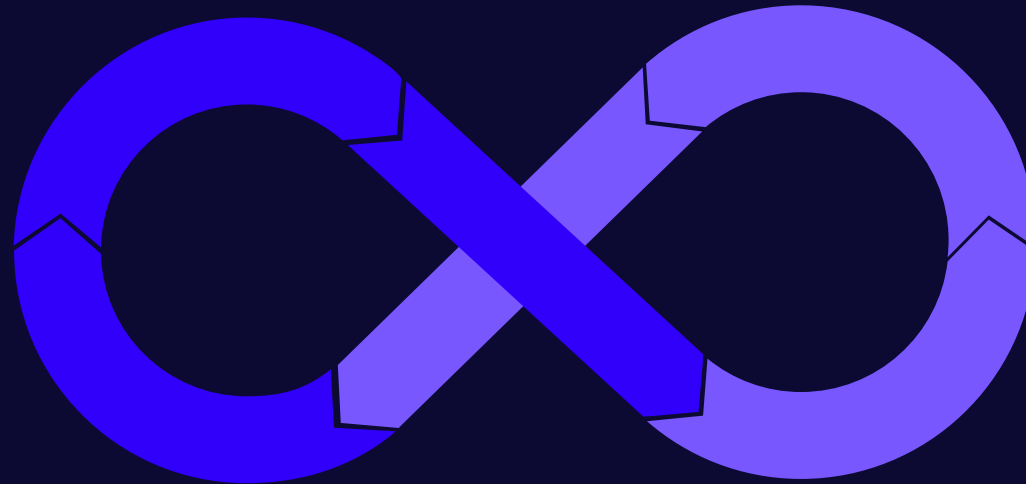
Resultado práctico

Las redes pueden reconocer imágenes, traducir texto o detectar fraude con alta precisión.

Entrenamiento: la función de pérdida

Predecir

Medir error
(Loss)

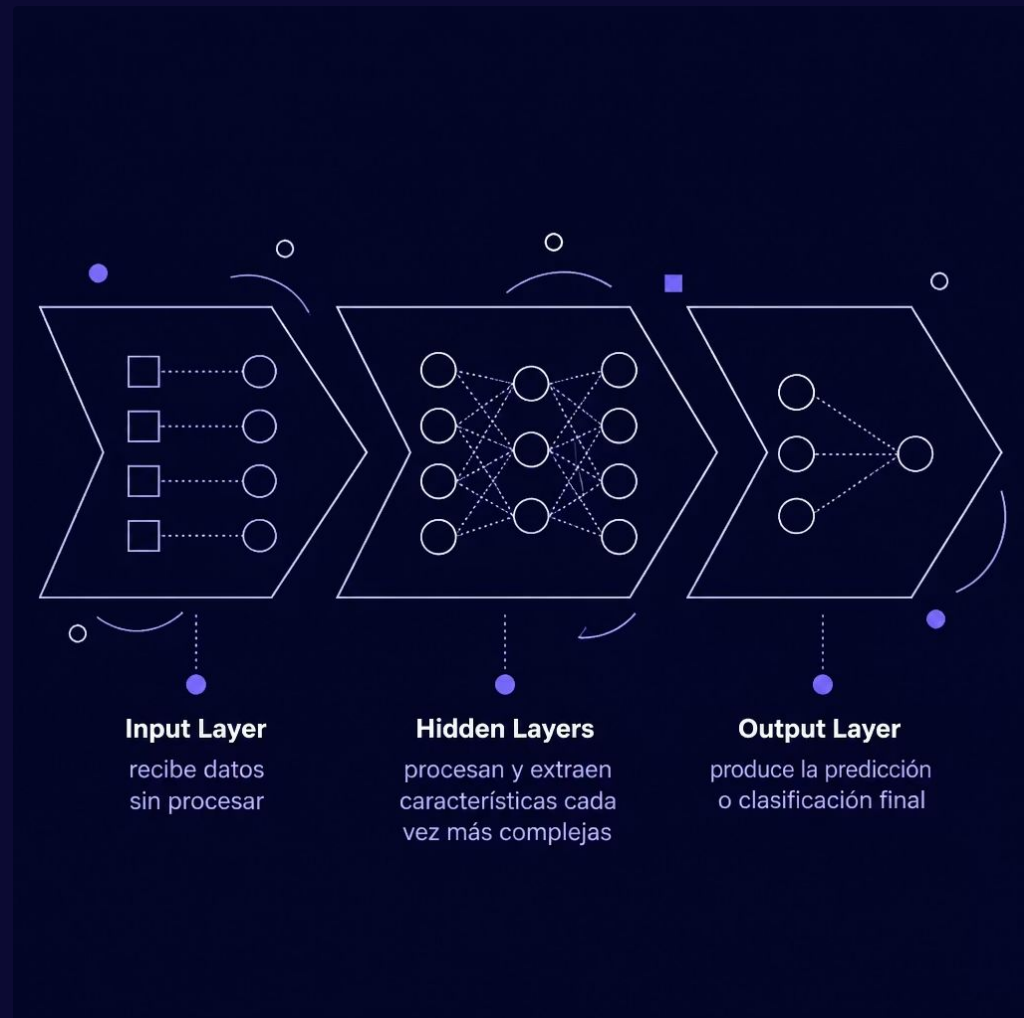


Repetir

Ajustar pesos y
bias

La **Loss Function** mide cuán equivocada está la red. El objetivo del entrenamiento es **minimizar esa pérdida** ajustando los pesos y el bias en cada iteración.

Estructura de una red: capas



Cada capa tiene un rol

- **Capa de entrada:** recibe los datos brutos
- **Capas ocultas:** extraen características complejas
- **Capa de salida:** produce la predicción final

La potencia surge de combinar **muchas neuronas en múltiples capas.**

¿Qué es la profundidad de una red?

Más capas ocultas = más profundidad

Una red más profunda puede aprender patrones más complejos y abstractos.

Pero tiene un coste

Mayor demanda computacional, necesita más datos etiquetados y aumenta el riesgo de overfitting.

Es una decisión de diseño

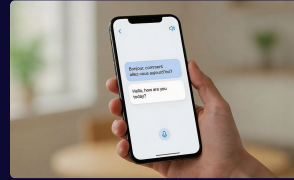
La arquitectura óptima depende siempre de la tarea y la cantidad de datos disponibles.

Aplicaciones de Redes Neuronales



Reconocimiento de imágenes

Reconocimiento facial, detección de objetos e imágenes médicas con CNN.



NLP: Lenguaje Natural

Traducción automática, análisis de sentimientos y generación de texto con RNN y Transformers.



Vehículos autónomos

Percepción del entorno, predicción de tráfico y control del vehículo en tiempo real.



Recomendaciones

Personalización de vídeo, compras y música basada en comportamiento del usuario.

Desafíos de las Redes Neuronales

Principales limitaciones

- Requieren **grandes recursos computacionales**
- Dependen de **datos etiquetados en gran cantidad**
- Alto riesgo de **overfitting** sin control

Estrategias para mitigarlos

- **Regularización:** restricciones que penalizan la complejidad excesiva del modelo
- **Transfer Learning:** reutilizar conocimiento aprendido en otra tarea similar, reduciendo datos necesarios

Bibliotecas de Python para ML y Redes



Scikit-Learn

ML clásico: regresión, clasificación, clustering, métricas y preprocesamiento.



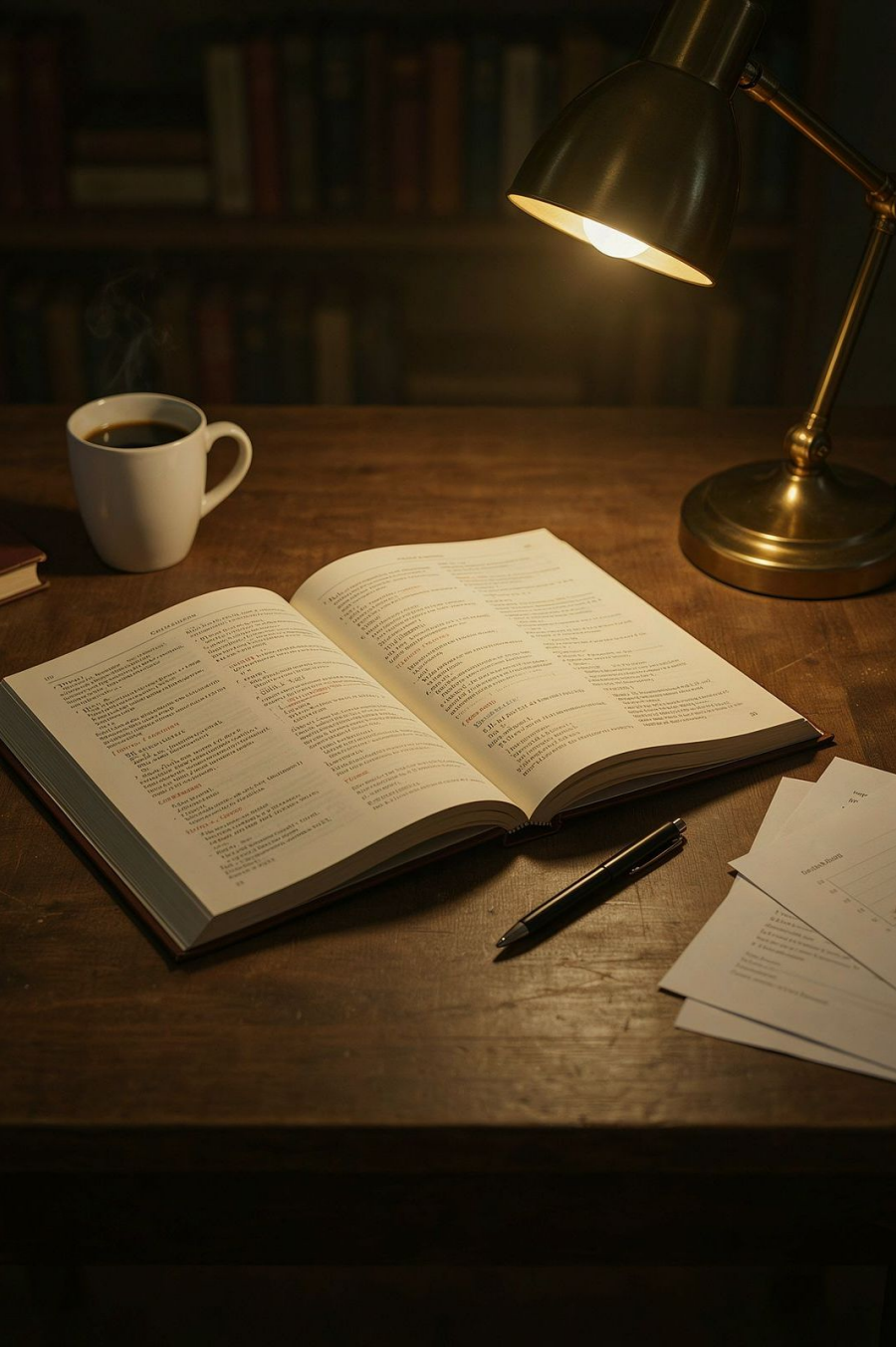
PyTorch

Framework moderno para construir y entrenar redes neuronales con flexibilidad total.



Microsoft Cognitive Toolkit

Herramienta de Microsoft optimizada para modelos de aprendizaje profundo a escala.



SCIKIT-LEARN

La documentación de Scikit-Learn

El puente entre la teoría y la práctica. Aprender a usarla bien es tan importante como conocer los algoritmos.

Cuatro secciones clave de la documentación



User Guide

Conceptos fundamentales, buenas prácticas, errores comunes y evaluación.



API Reference

Diccionario técnico: clases, funciones, parámetros y valores de retorno.



Examples

Código ejecutable aplicado a problemas reales y datasets conocidos.



Tutorials

Flujos paso a paso, prácticos, desde datos hasta evaluación del modelo.

Flujo recomendado para usar la documentación



Identificar

Elegir tarea

Explorar

Comprender

No hace falta memorizar. La clave es saber **dónde buscar** y cómo interpretar lo que encuentras.

Parámetros clave: DecisionTreeClassifier

Parámetros principales

- `criterion` → cómo medir la calidad de cada división (ej. "gini")
- `max_depth` → profundidad máxima del árbol, controla complejidad
- `min_samples_split` → muestras mínimas para dividir un nodo

¿Por qué importan?

Modificar estos parámetros cambia significativamente el comportamiento del modelo. La documentación explica cada uno con ejemplos claros.

- ❏ Cada modelo tiene sus propios parámetros. Siempre consulta la API Reference antes de ajustarlos.

Buscar la mejor configuración de parámetros

1

No probar al azar

La exploración manual es ineficiente y poco reproducible.

2

Grid Search

Prueba sistemáticamente todas las combinaciones de parámetros definidas.

3

Random Search

Muestrea configuraciones aleatoriamente — más rápido en espacios grandes.

4

Evaluar y comparar

Seleccionar la configuración con mejores métricas sobre el test set.

Métricas según el tipo de tarea

Clasificación

- **Accuracy:** porcentaje de aciertos totales
- **Precision:** de los que predije positivos, ¿cuántos lo eran?
- **Recall:** de los positivos reales, ¿cuántos capturé?
- **F1 Score:** media armónica de Precision y Recall

Regresión

- **MSE:** error cuadrático medio ↓
- **R²:** varianza explicada por el modelo ↑

Regla importante

La evaluación es **continua**: Entrenar → Evaluar → Ajustar
→ Nuevos datos → Volver a evaluar.



PRÁCTICA

Ejemplo práctico: Dataset Iris

El dataset Iris clasifica flores en 3 especies según medidas de sépalo y pétalo. Es el "Hola Mundo" del Machine Learning supervisado.

Flujo completo con Iris



Cargar Iris

Dividir datos

Modelo
Árbol

Entrenar

Predecir

Al final también puedes **visualizar el árbol de decisión** para interpretar exactamente qué criterios usa el modelo para clasificar cada flor.

Código completo: clasificación con Iris

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

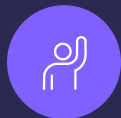
```
iris = load_iris()
X = iris.data
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
modelo = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    min_samples_split=10
)
modelo.fit(X_train, y_train)
```

```
y_pred = modelo.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

¿Qué aprendemos de la documentación?



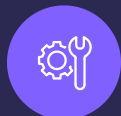
QUÉ algoritmo usar

Según el tipo de tarea y los datos disponibles



CÓMO funciona

Fundamentos matemáticos y supuestos del modelo



QUÉ parámetros tiene

Opciones de configuración y valores por defecto



CÓMO evaluarlo

Métricas apropiadas y cómo interpretarlas

No hay que memorizar todo: hay que saber **dónde buscar** y **cómo experimentar**.

Preguntas clave del módulo

1

¿Qué analogía describe mejor el ML?

El mapa del tesoro: datos = pistas, algoritmo descubre el patrón, modelo = resultado, predicciones = tesoro.

2

¿Para predecir clic Sí/No en recomendaciones?

Regresión Logística:
clasificación binaria con probabilidades.

3

¿Paso crucial antes de modelar precios de coches?

Limpiar y preparar los datos:
gestionar valores faltantes e incoherencias.

Más respuestas clave del módulo

¿Qué hace la función sigmoidea?

Transforma la combinación lineal en una **probabilidad entre 0 y 1**, permitiendo que la Regresión Logística clasifique.

¿Para qué sirve una Red Neuronal?

Imita la estructura del cerebro humano para **aprender patrones de datos y hacer predicciones complejas**.

¿Predecir compras con historial etiquetado?

Aprendizaje supervisado: hay datos etiquetados disponibles para entrenar el modelo.

Ideas clave para recordar

Regresión Lineal

Predice valores continuos. Evalúa con **MSE** ↓ y **R²** ↑.

Regresión Logística

Predice probabilidades y las convierte en clases. Clave: **sigmoidea + threshold**.

Redes Neuronales

Neuronas → capas → entrenamiento por pérdida.
Potentes pero costosas.

Scikit-Learn

No memorices: **aprende a buscar, probar y evaluar** con la documentación.

"En Machine Learning no se trata solo de entrenar modelos: se trata de **preparar bien los datos**, elegir el modelo adecuado, medir con las métricas correctas y ajustar usando documentación y experimentación."

¡Ahora tienes las bases para construir tus primeros modelos!

